DOCUMENTO TÉCNICO DE INGENIERÍA DE SOFTWARE

Plan de Acción: Proyecto SGA-DO

> **Propósito del Sistema:** El objetivo principal es construir un programa interactivo y robusto que permita gestionar estudiantes y calificaciones de una manera muy sencilla, evitando errores comunes durante la captura y procesamiento de datos académicos.

Puntos Clave y Objetivos del Sistema

1 Menú Interactivo

Interfaz en consola con opciones claras: registro de alumnos y profesores, asignación de notas, deshacer registros, generación de colas de certificados y salida segura del sistema.

2 Sistema a Prueba de Fallos

Implementación de validaciones con bloques `try-catch` para evitar caídas. Se notificará explícitamente: *"Error: Ingrese un valor numérico válido"* ante entradas no numéricas.

3 Función Deshacer (LIFO)

Mecanismo para revertir equivocaciones en la captura de notas mediante una **Pila (Stack)**, permitiendo revertir al estado anterior con "Deshacer Último Registro de Nota".

4 Certificados Automáticos

Barrido integral de estudiantes aprobados para generar el archivo `certificados_pendientes.txt` de forma secuencial y justa empleando una **Cola FIFO**.

5 Persistencia Automática

Garantía de almacenamiento íntegro e inmediato. Toda modificación se reflejará automáticamente en los archivos persistentes `alumnos.txt` y `profesores.txt` .

6 Polimorfismo Académico

Arquitectura orientada a objetos con clase base `ProgramaAcademico` y método virtual/polimórfico `evaluarAprobacion` adaptado a Curso, Diplomado y Bootcamp.

> **Arquitectura Orientada a Robustez:** El diseño contempla la desacoplación de responsabilidades entre la captura de interfaz de usuario, la validación de tipos primitivos, el procesamiento lógico de notas y el subsistema de persistencia en disco.

ECOSISTEMAS DE IMPLEMENTACIÓN

Tecnologías a Utilizar y Estrategia de Aplicación

Para evidenciar dominio integral en múltiples paradigmas y plataformas, la lógica del sistema SGA-DO se construirá en tres lenguajes distintos, explotando las capacidades nativas de cada entorno:

Python 3 (Lenguaje Principal)

BASE DE REFERENCIA

Construcción de la versión base aprovechando su sintaxis limpia y soporte completo para Programación Orientada a Objetos (POO). Empleo de **listas dinámicas nativas** para modelar el comportamiento exacto de la Pila (LIFO) y la Cola (FIFO) sin dependencias ni librerías externas adicionales.

Java (Versión Estricta Orientada a Objetos)

ESTRICTO TIPADO & POO

Replicación bajo estándares empresariales: estricto encapsulamiento (`private`), colecciones oficiales de la API estándar (`Stack` y `LinkedList/Queue`), y gestión estructurada de excepciones de entrada/salida (`IOException`) y parseo de datos.

C++ (Versión Compilada de Alto Rendimiento)

GESTIÓN MANUAL

Implementación a bajo nivel con control explícito del ciclo de vida en memoria. Creación manual de estructuras dinámicas de Pila y Cola mediante **Nodos y punteros (`new` / `delete`)**, garantizando cero fugas de memoria (*Memory Leaks*) con destructores auditados.

Almacenamiento I/O Nativo (.txt)

PERSISTENCIA INMEDIATA

Serialización mediante formato de valores separados por comas (CSV) en ficheros planos. Toda operación de alta, baja o actualización desencadena sincronización instantánea con el disco para asegurar persistencia transaccional y coherencia de datos.

Lenguaje	Estructura Pila (LIFO)	Estructura Cola (FIFO)	Gestión de Memoria
Python 3	Lista nativa (`append/pop`)	Lista nativa (`pop(0)`)	Garbage Collector automático
Java	`java.util.Stack`	`Queue / LinkedList`	JVM Garbage Collector
C++	Nodos enlazados manuales	Nodos enlazados manuales	Manual con `delete` en destructores